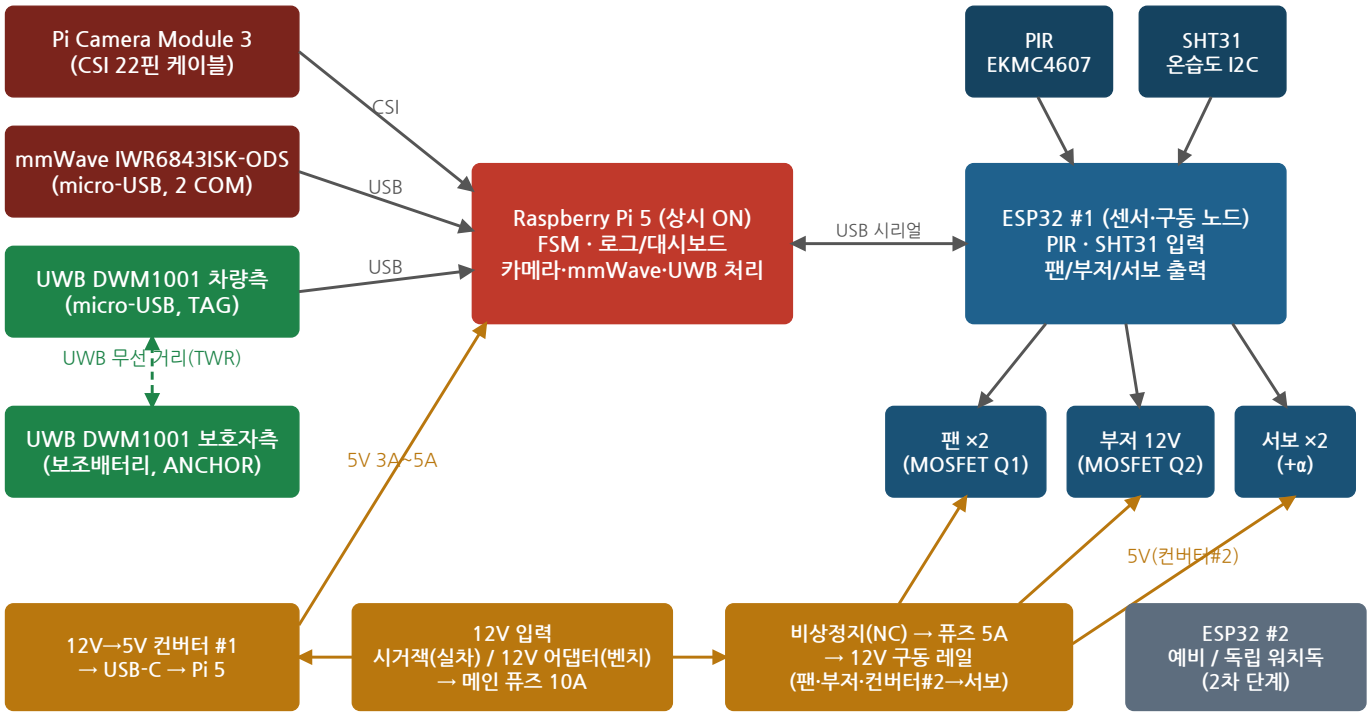
SafeGuard 하드웨어 연결 가이드

UWB · PIR/Camera · mmWave · Thermal · Actuator — 3인 분담 조립 매뉴얼

v1.0 | 2026-09-12 | 제24회 임베디드SW경진대회 자동차/모빌리티 부문 | 팀 두고내리지말아조

<그림 1> SafeGuard 전체 하드웨어 연결 개요



이 문서의 목적 — 소프트웨어 작업 이전에 **모든 부품이 전기적으로 올바르게 연결되고, 부품별 단독 동작이 확인된 상태**를 만드는 것. 블록 A/B/C로 나누어 3명이 병렬로 진행하고, 마지막 통합 단계는 다 같이 진행한다.

3가지 대원칙

1. **Pi 5는 상시 구동, 나머지는 이벤트 기반 ON/OFF.** 심사 피드백대로 완전한 전원차단 대신 '이벤트 기반 센서·연산 활성화 구조'로 성과를 한정한다.
2. **12V 구동부와 5V/3.3V 로직부를 물리적으로 분리하고 GND만 공통.** 3.3V 핀에 12V가 닿는 순간 Pi/ESP32는 죽는다.
3. **블록별로 따로 만들고, 각각 검증한 뒤, 마지막에 합친다.** 한 번에 다 꽂고 안 되면 원인을 못 찾는다.

목차

장	제목	내용
1	전체 그림: 무엇이 어디에 연결되는가	Pi 5 vs ESP32 역할 구분, 심사 피드백 반영 포인트
2	3개 블록으로 나누기와 역할 분담	분담 vs 공동 작업 판단, 일정, 완료 조건
3	0단계 (다같이): 개봉·인벤토리·추가 구매 확인	빠진 케이블/부품 목록, 안전 수칙
4	블록 A (담당 1): 전원부 + 구동부 + ESP32 센서 노드	전원 트리, MOSFET 회로, ESP32 핀맵, PIR/SHT31/서보/비상정지
5	블록 B (담당 2): Raspberry Pi 5 + 카메라 + mmWave	OS 설치, 카메라 케이블, IWR6843ISK-ODS 플래싱/UART
6	블록 C (담당 3): UWB DWM1001-DEV 2노드 거리 측정	TAG/ANCHOR 설정, 보조배터리, 5m 이탈 테스트
7	4단계 (다같이): 통합 연결과 전원 투입 순서	연결 순서, 통합 체크리스트, 필수 시연 리허설, 소비전력 측정
부록	핀맵 총괄표 · 부품 인벤토리 체크표 · 문제 해결 FAQ · 참고 링크	

1. 전체 그림: 무엇이 어디에 연결되는가

표지의 <그림 1>이 전체 연결도다. 핵심은 **두뇌(Pi 5)**, **손발(ESP32 #1)**, **전원**, 그리고 **보호자가 들고 다니는 UWB 노드**의 4덩어리로 보는 것이다.

1.1 Pi 5와 ESP32의 역할 구분 (심사위원 요청: 'ESP에서 뭐하고 라즈베리파이에서 뭐하고 구분')

장치	전원	담당 기능	연결되는 부품
Raspberry Pi 5 (16GB)	상시 ON (27W 어댑터 / 실차: 12V→5V 컨버터#1)	FSM(상태기계) 실행, 카메라 프레임 차분, mmWave UART 파싱, UWB 거리 파싱, 온도 예측(Thermal RC), 로그/대시보드, ESP32에 명령 전송	Camera Module 3 (CSI), IWR6843ISK-ODS (USB), DWM1001 차량측 (USB), ESP32 #1 (USB)
ESP32 #1 (센서·구동 노드)	Pi USB에서 5V 급전 (벤치 단독 테스트는 노트북 USB)	저전력 센서 읽기(PIR, SHT31) → JSON으로 Pi에 전송, Pi 명령을 받아 팬/부저/서보 구동, 비상정지 상태 감시, (2차) PIR 인터럽트 딥슬립 웨이크업	PIR EKMC4607112K, SHT31, IRLZ44N ×2 (팬, 부저), DS3218 서보 ×2, 비상정지 보조 접점
ESP32 #2	-	1차 단계에서는 예비. 2차 단계에서 '독립 워치독'(Pi 하트비트 감시 → 무응답 시 부저 강제 ON)으로 사용 가능	-
DWM1001-DEV #1 (차량측)	Pi USB	UWB TWR 거리 측정값을 시리얼로 출력 (TAG 역할, 6장 참고)	Pi 5 USB
DWM1001-DEV #2 (보호자측)	10,000mAh 보조배터리	보호자가 소지, 차량측 노드와 거리 측정 (ANCHOR 역할)	보조배터리만

1.2 심사 피드백이 하드웨어 결정에 반영된 곳

심사 피드백	하드웨어에서의 결정
Raspberry Pi는 상시 구동, 나머지는 이벤트 기반 ON/OFF	Pi 전원 분기(컨버터#1)는 비상정지 뒤에 두지 않고 항상 살려둔다. 카메라/mmWave/PIR은 소프트웨어에서 활성·비활성 제어(전원 차단 회로는 1차 범위 밖).
전원 차단 회로와 wake-up을 소자별로 설명 필요	1차: ESP32는 상시 급전 + PIR 인터럽트로 딥슬립 웨이크업(소프트웨어). 팬/부저는 MOSFET로 실제 전원 스위칭. 2차(선택): 카메라/mmWave USB 전원 스위칭.

심사 피드백	하드웨어에서의 결정
각 단계 소비전력·전환시간 측정	테스터기(DM-1010) 10A 단자로 12V 입력 전류를 직렬 측정. 7.6절 측정표 템플릿 사용. USB 전력측정기 추가 구매 권장.
보호자 측 Tag도 독립 전원 필요	DWM1001 #2는 보조배터리(10,000mAh)로 급전. 저전류 자동 차단 문제는 6.3절에서 검증.
창문 개방(서보)은 플러스 알파, 메인은 FSM 안정 동작	서보는 컨버터#2로 분리 급전하여 실패해도 메인 회로에 영향 없게 구성. 통합 체크리스트에서 우선순위 '선택'으로 표시.
과전류 보호·비상정지 구체화	메인 퓨즈 10A + 구동 분기 퓨즈 5A, 비상정지(NC 접점)는 12V 구동 레일에 직렬. Pi는 살아서 로그를 남긴다.
안정적 시연, 무리한 조건 금지	12V 부하는 모두 MOSFET 1채널당 1개 기능으로 단순화. 릴레이 대신 MOSFET(무접점, 소음 없음).

2. 3개 블록으로 나누기와 역할 분담

2.1 블록 정의

블록	담당	포함 부품 (엑셀 No.)	필요한 도구	결과물(완료 조건)
A. 전원부 + 구동부 + ESP32 센서 노드	담당 1	ESP32 ×2(13), PIR(12), SHT31(11), IRLZ44N ×2(31), 1N5819(15), 저항키트(33), 팬 120mm(17), 팬 108F(18), 부저(14), 서보 ×2(10), 비상정지(19), 퓨즈·홀더(29,30), 컨버터 ×2(34), 시거잭(20), 만능기판(6), 납땜세트(5), 와이어(24), JST(26), 수축튜브(8), 매트(9), 확대경(7), 테스터기(16)	납땜인두, 테스터기, 노트북(Arduino IDE)	① 12V 투입 시 5V 두 계통 정상 ② 비상정지 누르면 구동 레일 0V ③ ESP32 시리얼에 PIR/SHT31 값 출력 ④ 시리얼 명령으로 팬·부저·서보 ON/OFF ⑤ 소비전류 측정표 작성
B. Pi 5 + 카메라 + mmWave	담당 2	Pi 5(1), microSD(28), 27W 어댑터(21), Camera Module 3(2), IWR6843ISK-ODS(3), 스탠드오프(27)	노트북(Windows: Pi Imager, TI UniFlash, mmWave Visualizer), micro-USB 데이터 케이블	① Pi 부팅+SSH ② rpicam으로 사진 저장 ③ mmWave 데모 펌웨어 플래싱 후 Visualizer에서 포인트 클라우드 확인 ④ Pi에서 /dev/ttyACM로 mmWave 데이터 수신
C. UWB 2노드 거리 측정	담당 3	DWM1001-DEV ×2(4), 보조배터리(35), micro-USB 케이블(36)	노트북(Tera Term/PuTTY), 안드로이드폰(DRTLS 앱, 선택), 줄자	① 두 노드 같은 네트워크로 설정 ② 차량측 노드 셸에서 거리 출력 ③ 1/3/5/7m 거리 오차표 ④ 보조배터리로 10분 이상 꺼지지 않음 ⑤ Pi에서 거리 파싱 스크립트 동작

2.2 분담할까, 다같이 할까? → 블록 작업은 분담, 0단계와 통합은 다같이

분담을 추천하는 이유

- 세 블록은 통합 전까지 서로 전기적으로 **완전히 독립**이다. A는 12V와 인두, B는 노트북과 TI 툴, C는 노트북과 줄자. 같은 자리에서 서로 기다릴 일이 없다.
- 병렬로 하면 소요 시간이 대략 1/3로 줄고, 각자가 자기 블록의 '전문가'가 되어 통합 때 디버깅이 빨라진다.
- 블록 A가 손이 가장 많이 가므로(납땜·회로) 가장 손재주 있는 사람 또는 2명이 A를 맡고, B와 C는 1명이 순차로 맡는 조합도 가능하다.

반드시 다같이 해야 하는 것

- **0단계 인벤토리**: 36개 품목을 함께 개봉하고 누락·파손을 확인한다 (30분~1시간).

- **4단계 통합**: GND 공통, USB 3개 연결, 전원 투입 순서. 여기서 문제가 나면 세 블록 지식이 다 필요하다.
- **필수 시연 리허설**: UWB 거리 증가 → PIR/카메라 → mmWave → 온도 → 팬/부저를 한 흐름으로 돌린다.

2.3 권장 일정

단계	기간	형태	내용	산출물
0단계	반나절	다같이	개봉, 인벤토리 체크(부록 B), 추가 구매 발주(3.2절), 작업 공간 세팅	체크표, 추가 구매 목록
1단계	2~3일	분담	블록 A/B/C 각자 진행. 매일 저녁 팀 채팅에 '완료 조건 표 + 사진/로그' 공유	블록별 완료 조건 표
2단계	반나절	다같이	블록 A 만능기판 이관 검수(다른 사람이 테스터기로 재확인), 케이블 길이 확정	검수 사진
3단계	1일	다같이	통합 연결(7장), 전원 투입, 통합 체크리스트	통합 체크리스트
4단계	반나절	다같이	필수 시연 시나리오 하드웨어 리허설, 소비전력 측정표 작성	측정표, 시연 영상

참고 각 블록의 담당자는 작업 중 **배선 사진을 매 단계 찍어** 둔다. 통합 때 '이 선이 어디 갔더라'가 가장 많은 시간을 잡아먹는다.

3. 0단계 (다같이): 개봉·인벤토리·추가 구매 확인

3.1 인벤토리 체크

부록 B의 36개 품목 체크표를 프린트해서 개봉하면서 체크한다. 각 품목을 블록 A/B/C 상자로 바로 분류해 두면 이후 찾는 시간이 없다.

3.2 추가 확인·구매가 필요한 항목 (지금 바로 확인할 것)

엑셀 리스트만으로는 연결이 안 되는 부분이 있다. 아래는 조립 중 막히는 대표 항목이다. 대부분 소액이며 하루라도 빨리 주문해야 1단계가 안 멈춘다.

항목	왜 필요한가	확인 방법 / 대안	예상 비용
Pi 5용 카메라 케이블 (22핀 mini ↔ 15핀)	Camera Module 3에 동봉된 케이블은 Pi 4용 15핀. Pi 5의 CSI 커넥터는 22핀 소형이라 **안 꽂힌다**.	박스 안에 얇은 22핀 케이블이 같이 있는지 확인. 없으면 디바이스마트 'Raspberry Pi 5 카메라 케이블(Standard-Mini)' 200~300mm 구매.	~3,000원
micro-USB 데이터 케이블 총 4개	mmWave(1), UWB 차량측(1), ESP32(1), UWB 보호자측↔보조배터리(1). 현재 1개(No.36)만 있음.	집에 있는 케이블 재활용 가능하나 **충전 전용(데이터선 없음) 케이블은 인식 안 됨**. 노트북에 꽂아 장치 인식되는지로 판별.	개당 1,000~2,000원
벤치용 12V DC 어댑터 (12V 5A, 5.5×2.1mm) + DC잭 터미널	시거잭 충전기는 실차 전용. 실내에서 12V 팬·부저·컨버터를 돌릴 12V 소스가 리스트에 없다.	12V 5A 어댑터 + 'DC잭 → 스크류 터미널' 변환 어댑터(암).	~12,000원
브레드보드 + 점퍼선	만능기판 납땜 전에 회로를 검증할 임시 보드. 기존 Pi 4B 프로토타입 빵판·점퍼 재활용.	기존 보유분 확인. 없으면 830홀 브레드보드 1개 + M-M/M-F 점퍼 세트.	~8,000원
Pi 5 액티브 쿨러 (권장)	카메라 프레임 차분 + mmWave 파싱 + 대시보드 동시 구동 시 발열로 클럭 저하. 모형차 안은 통풍이 나쁘다.	라즈베리파이 공식 Active Cooler.	~8,000원
USB 전력 측정기 (선택)	심사 질문 '소비전력을 뭐로 비교하나'에 대한 답. 각 USB 장치(mmWave, ESP32, UWB)의 전류를 개별 측정 가능.	USB-A 인라인 전압·전류계.	~5,000원

항목	왜 필요한가	확인 방법 / 대안	예상 비용
컨버터(No.34) 출력 형태 확인	제품이 USB-A 포트 출력형이면 서보 급전용 5V/GND 선을 뽑을 수 없다.	박스 열어 확인. USB형이면 USB-A 케이블 하나를 잘라 빨강(5V)/검정(GND)을 사용.	0원
팬 커넥터 확인	120mm LED 팬은 3핀/4핀 Molex, 108F는 2선. MOSFET 회로에는 +/ 만 필요.	커넥터를 잘라 JST 커넥터로 재단(JST 키트 No.26 활용). 노란/파란 3·4번째 선(RPM/PWM)은 절연 후 미사용.	0원
서보 전원 커넥터 (JST 또는 터미널)	DS3218 배선은 듀폰 3핀. 5V 전원선은 별도 배선 필요.	JST 키트로 5V/GND 분기 케이블 제작.	0원

3.3 안전 수칙 (인쇄해서 작업대에 붙일 것)

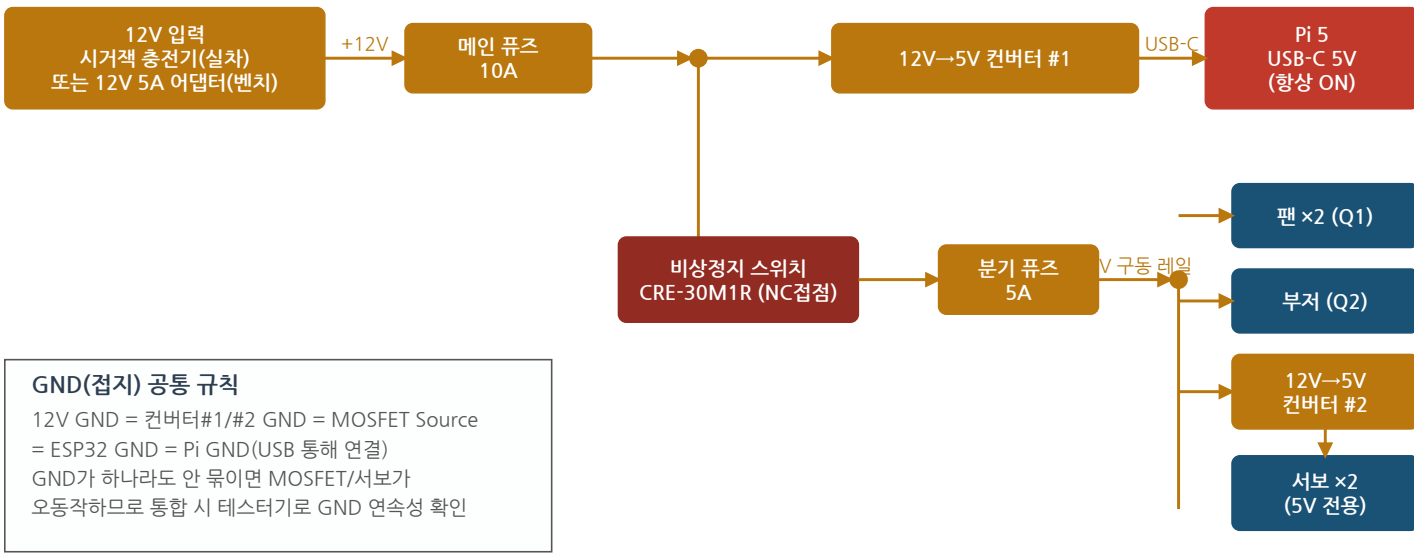
- **배선은 항상 전원 OFF 상태에서.** 전원을 넣기 전 테스터기 저항 모드로 **+12V-GND 사이가 쇼트(0Ω)가 아닌지** 확인한다.
- **3.3V/5V 핀에 12V 절대 금지.** 12V가 다니는 선은 빨간색, 5V는 주황/노랑, GND는 검정으로 색을 고정한다(3색 실리콘 와이어 활용).
- **서보는 ESP32/Pi의 5V 핀에서 급전 금지.** DS3218은 순간 2A 이상을 먹어 보드가 리셋되거나 타 버린다. 반드시 컨버터#2에서.
- **MOSFET/다이오드 극성 확인 후 납땜.** 1N5819의 띠(캐소드)가 +12V 쪽. 거꾸로 달면 전원 넣는 순간 퓨즈가 나간다(퓨즈가 제 역할을 한 것).
- **인두는 절연 매트 위에서, 수축튜브는 납땜 전에 먼저 끼운다.** (납땜 후 끼우려면 늦다)
- **Pi 5는 반드시 shutdown 후 전원 분리.** SD카드 손상 방지. 카메라 케이블은 Pi 전원 OFF에서만 탈착.

4. 블록 A (담당 1): 전원부 + 구동부 + ESP32 센서 노드

가장 손이 많이 가는 블록. **브레드보드에서 먼저 검증하고, 되는 것을 확인한 뒤에만 만능기판에 납땜**한다. 순서를 지키면 부품을 태울 일이 거의 없다.

4.1 전원 트리

〈그림 2〉 전원 트리 (블록 A)

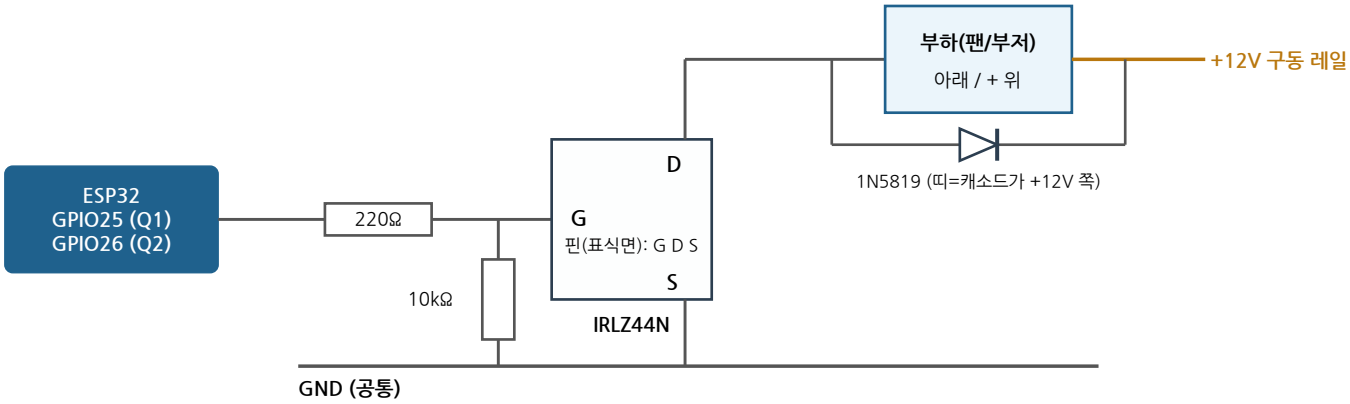


비상정지는 구동 레일만 끊고 Pi/ESP32 전원은 유지 → 사고 시 로그가 남음

구간	부품	규격/설정	비고
12V 입력	시거잭 충전기(No.20) / 12V 5A 어댑터(추가 구매)	12V, 실차 최대 10A 시거 소켓	시거잭 충전기가 USB 출력형이면 12V 원시 출력이 없을 수 있음 → 실차 시 시거 소켓 '분배기'의 12V 단자 또는 배터리 직결(퓨즈 필수) 검토
메인 퓨즈	퓨즈홀더(No.30) + 카퓨즈 10A(No.29)	블레이드 퓨즈 10A	12V 입력 직후 +선에 직렬
분기 1 → Pi	12V→5V 컨버터 #1(No.34)	출력 5V 10A, USB-A → USB-C 케이블 → Pi 5	비상정지 영향 없음. Pi는 항상 ON
분기 2 → 구동 레일	비상정지 CRE-30M1R(No.19) NC접점 → 퓨즈홀더 + 5A	안 눌렀을 때 통전, 누르면 차단	테스터기 부저 모드로 NC/NO 접점 미리 판별 (4.7절)
구동 레일 부하	팬 120mm(No.17) + 팬 108F(No.18) → Q1, 부저 ×2 병렬 또는 1개(No.14) → Q2	팬 합계 약 0.5A, 부저 약 30mA	부하 선은 MOSFET 드레인으로, +선은 레일로
분기 2-2 → 서보	12V→5V 컨버터 #2(No.34)	5V, 서보 2개 순간 최대 4~5A	서보 신호선만 ESP32, 전원은 여기서. GND 공통

4.2 MOSFET 스위치 회로 (IRLZ44N ×2)

〈그림 3〉 로우사이드 MOSFET 스위치 회로 (채널당 1개, 총 2개)



동작: GPIO HIGH(3.3V) → 게이트 ON → 부하 단자가 GND로 → 부하 동작. GPIO LOW → OFF. Q1=팬 2개 병렬, Q2=부저.

- IRLZ44N은 **로직 레벨** MOSFET이라 ESP32의 3.3V로도 충분히 켜진다(팬 0.5A 수준에서 발열 거의 없음).
- 게이트 직렬 220Ω(저항키트에서 선택, 100~330Ω 아무거나)과 게이트-소스 10kΩ 풀다운은 **부팅 중 GPIO가 떠 있을 때 팬이 멋대로 도는 것**을 막는다.
- 1N5819 프리휠 다이오드는 팬 모터의 역기전력에서 MOSFET을 보호한다. 부저 채널에도 같은 구성으로 달아 두면 안전하다(5개 있음).
- 핀 배열: IRLZ44N 정면(표식면)에서 왼쪽부터 **G(게이트) - D(드레인) - S(소스)**. 방열탭은 드레인과 연결되어 있으니 두 MOSFET 탭이 서로 닿지 않게 한다.

4.3 ESP32 #1 핀맵 (DevKitC WROOM-32D)

기능	ESP32 핀	상대 부품 핀	전압	메모
PIR 출력	GPIO27	PIR OUT	3.3V 로직	RTC GPIO → 2차 단계 딥슬립 ext0 웨이크업 가능
SHT31 SDA	GPIO21	SDA	3.3V	I2C 기본 핀
SHT31 SCL	GPIO22	SCL	3.3V	I2C 기본 핀. 주소 0x44 (ADDR 핀 GND) / 0x45 (VCC)
팬 스위치 Q1	GPIO25	Q1 게이트 (220Ω 경유)	3.3V	HIGH=팬 ON
부저 스위치 Q2	GPIO26	Q2 게이트 (220Ω 경유)	3.3V	HIGH=부저 ON

기능	ESP32 핀	상대 부품 핀	전압	메모
서보 1 신호	GPIO18	서보 주황/노랑	3.3V PWM 50Hz	DS3218은 3.3V 신호 인식됨
서보 2 신호	GPIO19	서보 주황/노랑	3.3V PWM 50Hz	플러스 알파 기능
비상정지 상태 감시(선택)	GPIO32 (INPUT_PULLUP)	비상정지 보조 접점(NO) 한쪽, 다른 쪽 GND	3.3V	눌리면 LOW → Pi에 'ESTOP' 이벤트 전송
상태 LED	GPIO2 (온보드)	-	-	하트비트 깜빡임
3.3V 출력	3V3	PIR VCC, SHT31 VCC	3.3V	합계 수 mA, 문제없음
GND	GND (여러 개)	PIR/SHT31 GND, Q1·Q2 소스, 서보 GND, 12V GND	-	**모든 GND 공통**
USB	micro-USB	Pi 5 USB-A	5V	전원 + 시리얼(CP2102 → Pi에서 /dev/ttyUSB0)

주의 ESP32에서 **쓰면 안 되는 핀**: GPIO0/2/12/15(부팅 스트래핑, 2는 LED만), GPIO6~11(플래시), GPIO34~39(입력 전용, 풀업 없음). 위 표의 핀만 쓰면 안전하다.

4.4 PIR 센서 (SparkFun Breakout, Panasonic EKMC4607112K)

- 핀 3개: **VCC → 3V3, GND → GND, OUT → GPIO27**. 5V가 아니라 3.3V로 급전한다.
- 전원 투입 후 **약 30초 워밍업** 동안 출력이 불안정하다. 테스트할 때 30초 기다렸다가 손을 흔든다.
- 출력 확인법: 테스터기 DC 전압으로 OUT-GND를 재면서 손을 흔든다. 감지 시 약 3.3V(HIGH)로 뜨면 코드에서 **HIGH=감지**. 반대로 평소 HIGH·감지 시 LOW라면 코드에서 반전(**LOW=감지**)하고 ESP32 내부 풀업을 켠다. 실측으로 확정하고 표에 적어 둔다.
- 감지 범위 최대 12m·넓은 시야각이므로 모형차 안에서는 **창밖 사람도 잡는다**. 렌즈 옆을 검은 테이프로 가려 뒷좌석만 보게 시야를 좁힌다(시연 안정성).

4.5 SHT31 온습도 센서 (I2C)

- 핀: **VCC → 3V3, GND → GND, SDA → GPIO21, SCL → GPIO22**. 대부분의 브레이크아웃은 풀업 저항이 내장되어 있어 외부 풀업 불필요.
- Arduino IDE에서 'Adafruit SHT31' 라이브러리 예제 → 시리얼 모니터에 온도/습도가 1초마다 뜨면 완료. 손으로 감싸 온도가 오르는지 확인.
- 설치 위치: 뒷좌석 헤드 높이, 직사광선·팬 바람이 직접 닿지 않는 곳. 팬 바로 앞에 두면 팬 ON 즉시 온도가 떨어져 예측 로직이 흔들린다.

4.6 서보 DS3218 ×2 (플러스 알파 기능)

- 선 3가닥: **갈색/검정 = GND, 빨강 = 5V(컨버터#2), 주황/노랑 = 신호(GPIO18/19)**.
- DS3218 정격 4.8~6.8V, 스톨 전류 2A 이상. **컨버터#2 5V에서 급전**하고 GND를 ESP32와 공통으로 묶는다. 신호선만 ESP32에 간다.
- ESP32Servo 라이브러리 스윕 예제로 0°↔180° 왕복 확인. 동작 중 컨버터#2 출력이 4.5V 밑으로 떨어지면 케이블이 가늘거나 접촉 불량.
- 우선순위는 '선택'이다. 메인 흐름(팬·부저)이 먼저 안정되기 전에는 서보에 시간을 쓰지 않는다.

4.7 비상정지 스위치 CRE-30M1R

- 테스터기 **부저(연속성) 모드**로 접점 두 단자를 잰다. **안 누른 상태에서 '삐'가 나고 누르면 끊기는 쌍이 NC 접점**이다. 이 NC 접점을 12V 구동 레일 +선에 직렬로 넣는다.
- 접점이 2쌍(NC+NO)이면 NO 쌍은 ESP32 GPIO32(풀업)-GND에 연결해 '눌림' 상태를 Pi에 보고한다. 1쌍뿐이면 이 기능은 생략.
- 판넬 장착형이라 모형차 외부(트렁크 뚜껑 등)에 구멍을 뚫어 고정. 시연 중 심사위원이 볼 수 있는 위치가 좋다.

4.8 작업 순서 (브레드보드 → 만능기판)

1. **ESP32 단독 인식**: 노트북 USB 연결 → Arduino IDE 보드 'ESP32 Dev Module', 포트 선택 → Blink(GPIO2) 업로드. 업로드 안 되면 BOOT 버튼을 누른 채 업로드 시작. CP2102 드라이버 필요 시 설치.
2. **PIR + SHT31 브레드보드**: 4.4/4.5절대로 연결 → 시리얼 모니터에 `{"pir":1,"temp":27.3,"hum":45.1}` 형태로 1초마다 출력되게 스케치 작성(이 형식이 나중에 Pi 파서의 입력이 된다).
3. **12V 레일 부하 없이 검증**: 퓨즈홀더-비상정지-컨버터#1/#2를 결선 → 테스터기로 12V-GND 쇼트 없음 확인 → 12V 투입 → 컨버터#1·#2 출력 5.0±0.2V 확인 → 비상정지 누르면 구동 레일 0V, 컨버터#1은 그대로 5V 확인.
4. **MOSFET 채널 1개 브레드보드**: <그림 3>대로 팬 1개만 연결 → ESP32 GPIO25 HIGH/LOW 코드로 팬 ON/OFF. 손으로 MOSFET 온도 확인(미지근 이하). 그다음 팬 2개 병렬, 그다음 Q2 부저.
5. **서보**: 컨버터#2 5V 급전, GPIO18 스윕. 서보 2개 동시 동작 시 5V 유지되는지 테스터기로 확인.
6. **시리얼 명령 프로토콜 통일**: Pi → ESP32로 `FAN 1`, `BUZ 0`, `SRV1 90` 같은 한 줄 텍스트 명령을 받아 동작하게 한다. 노트북 시리얼 모니터에서 타이핑으로 검증.
7. **만능기판 이관**: MOSFET 2개, 저항 4개, 다이오드 2~3개, JST 커넥터(12V IN, 팬, 부저, 서보 5V, ESP32 연결)를 100×100 양면 만능기판에 납땜. ESP32는 핀헤더 소켓으로 탈착 가능하게. 수축튜브로 절연.

8. **소비전류 측정**: 테스터기 10A DC 전류 모드를 12V 입력 +선에 직렬 → 대기 / 팬 ON / 부저 ON / 서보 동작 / 전체 각각 기록(7.6절 표).

4.9 블록 A 완료 조건 표 (담당자가 채워서 공유)

#	확인 항목	기대값	실측/결과	확인자
A1	12V 입력 → 컨버터#1 출력	5.0±0.2V		
A2	비상정지 누름 → 구동 레일 전압	0V (컨버터#1은 5V 유지)		
A3	PIR 손 흔들기 → 시리얼 pir 값	0→1 전환		
A4	SHT31 온도/습도	실내 온도 ±1°C		
A5	명령 FAN 1 / FAN 0	팬 2개 ON/OFF		
A6	명령 BUZ 1 / BUZ 0	부저 ON/OFF		
A7	명령 SRV1 0 / SRV1 180	서보 회전 (선택)		
A8	12V 입력 전류: 대기/팬/부저/서보/전체	mA 단위 5개 값		
A9	만능기판 이관 후 A1~A7 재확인	동일		

5. 블록 B (담당 2): Raspberry Pi 5 + 카메라 + mmWave

5.1 Pi 5 기본 세팅

1. 노트북에 **Raspberry Pi Imager** 설치 → OS: **Raspberry Pi OS (64-bit), Bookworm** → 저장소: 128GB microSD(No.28, 리더기 포함).
2. Imager의 '설정 편집'에서 호스트명(safeguard), 사용자/비밀번호, **Wi-Fi(팀 핫스팟)**, **SSH 활성화**를 미리 넣는다. 모니터 없이 SSH로 작업하기 위함.
3. SD 삽입 → 27W USB-C 어댑터(No.21) 연결 → 부팅 → 노트북에서 `ssh <user>@safeguard.local` 접속.
4. `sudo apt update && sudo apt full-upgrade -y` → `sudo raspi-config`에서 Interface Options: I2C, Serial Port(로그인 셸 No, 하드웨어 Yes) 활성화.
5. 실차에서 컨버터(비PD 전원)로 급전할 때 Pi 5는 USB 주변기기 전류를 600mA로 제한한다. mmWave+UWB+ESP32를 다 꽂으려면 `/boot/firmware/config.txt` 끝에 `usb_max_current_enable=1` 추가.
6. 스탠드오프(No.27)로 Pi를 아크릴 판에 고정하고, 액티브 쿨러가 있으면 장착.

```
# 부팅 후 확인 명령
vcgencmd measure_temp        # 온도
vcgencmd get_throttled       # 0x0 이면 전원 정상
ls /dev/ttyACM* /dev/ttyUSB*  # USB 시리얼 장치 목록
```

5.2 Camera Module 3 (CSI)

- **Pi 전원 OFF 상태에서** 작업. Pi 5에는 CAM/DISP 커넥터가 2개(CAM0, CAM1) 있고 둘 다 22핀 소형이다. 아무 쪽이나 사용(기본 CAM1 우선 인식).
- 3.2절의 **22핀↔15핀 케이블**이 필요. 커넥터의 갈색/검은 잠금 탭을 살짝 당겨 올린 뒤, 케이블의 **금속 접점면이 커넥터 고정부(접점) 쪽**, 파란 보강면이 잠금 탭 쪽을 향하게 끝까지 밀어 넣고 탭을 눌러 잠근다. 카메라 쪽(15핀)도 같은 원리.
- 부팅 후 확인:

```
rpicam-hello --list-cameras          # 카메라가 목록에 나오면 인식 OK
rpicam-still -o test.jpg -t 2000      # 사진 저장
rpicam-vid -t 5000 -o test.h264       # 5초 영상
# 노트북에서:  scp 사용자@safeguard.local:~/test.jpg .
```

- 목록에 안 나오면: 케이블 방향(가장 흔함), 잠금 탭이 덜 눌림, 케이블이 CAM이 아닌 DISP 라벨 쪽에 꽂힘을 순서대로 확인.
- 장착 위치: 룸미러 위치에서 뒷좌석을 향하게. 프레임 차분 방식이라 **흔들리지 않게** 고정하는 것이 오검출 방지의 핵심(벨크로 No.23 + 스탠드오프).

5.3 mmWave 레이더 IWR6843ISK-ODS

IWR6843ISK-ODS는 **단독(standalone) 모드**로 micro-USB 하나만 꽂으면 전원과 XDS110 디버거/UART가 모두 잡힌다. 별도 MMWAVEICBOOST 캐리어 보드는 필요 없다. USB를 꽂으면 PC/Pi에 **COM 포트 2개**가 생긴다: **User/CLI UART(115200)**는 설정 명령용, **Data UART(921600)**는 포인트 클라우드 출력용.

① 노트북(Windows)에서 먼저 살린다

1. TI 계정 생성 후 **UniFlash**, **mmWave SDK**(3.x, 6843 지원 버전), **mmWave Industrial Toolbox**(Resource Explorer에서 온라인 열람 가능), **mmWave Demo Visualizer**(웹 앱 + TI Cloud Agent) 설치.
2. 보드의 **SOP 스위치(S1)**를 'Flashing 모드'로 설정. 보드 실크 또는 TI 'IWR6843ISK-ODS User's Guide'의 SOP 표를 따른다(플래싱/기능 모드 두 조합만 기억하면 됨).
3. micro-USB 연결 → 장치 관리자에서 'XDS110 Class Application/User UART'와 'Auxiliary Data Port' COM 번호 확인.
4. UniFlash → 디바이스 IWR6843 → COM 포트(User UART) 선택 → 이미지: Industrial Toolbox의 **'Out of Box Demo (IWR6843ISK-ODS)'** 바이너리(.bin) 로드 → Load Image.

5. SOP를 'Functional 모드'로 변경 → USB 재연결(리셋) → mmWave Demo Visualizer에서 플랫폼 xWR6843, 안테나 ODS 선택, 두 COM 포트 지정 → 'Send Config to mmWave Device' → Plots 탭에서 **사람이 움직일 때 포인트 클라우드**가 뜨면 성공.
6. Visualizer에서 쓴 **.cfg 파일을 저장**해 둔다(Pi에서 같은 설정을 CLI UART로 보낼 때 사용).

② Pi 5에 연결

```
ls /dev/ttyACM*              # ttyACM0 = CLI(115200), ttyACM1 = Data(921600) 가 일반적
pip install pyserial
# cfg 파일을 한 줄씩 ttyACM0으로 전송 → ttyACM1에서 TLV 프레임 수신
# (TI 'mmWave Demo' 파이썬 파서 예제를 그대로 활용, 매직워드 0x0102 0x0304 0x0506 0x0708)
```

- Pi에 꽂았을 때 **반복 리셋/인식 불안정**이면 USB 전류 부족이다. 5.1절 usb_max_current_enable 설정 후에도 불안정하면 전원 공급형 USB 허브 사용.
- **어떤 랩을 쓸지**는 Out-of-Box로 재실이 확인된 뒤에 결정: 후보는 'Vital Signs'(호흡 확인, 보고서 서술과 일치) 또는 '3D People Counting'(ODS 오버헤드용). 하드웨어 단계에서는 Out-of-Box 포인트 클라우드까지만 목표.
- ODS 안테나는 **천장에서 아래를 보는 오버헤드 장착용**이다. 모형차 천장 중앙에 아래를 향하게 고정한다. 안테나 앞에 금속·아크릴 두꺼운 판이 없어야 함(얇은 아크릴은 통과 가능).
- 심사 피드백 대응: mmWave 장애 시 확정 조건은 소프트웨어 단계에서 정하되, 하드웨어에서는 USB 탈착 시 Pi가 `/dev/ttyACM` 소실을 감지할 수 있는지 미리 확인해 둔다(FAIL_SAFE 전이 입력).

5.4 블록 B 완료 조건 표

#	확인 항목	기대값	실측/결과	확인자
B1	SSH 접속 + get_throttled	0x0		
B2	rpicam-hello --list-cameras	imx708 (Camera Module 3) 표시		
B3	rpicam-still 사진	뒷좌석이 프레임에 들어옴		
B4	mmWave Out-of-Box 플래싱	Visualizer 포인트 클라우드		
B5	Pi에서 ttyACM0/1 인식 + 데이터 프레임 수신	매직워드 확인, 프레임/초 기록		
B6	USB 3개 동시 연결(ESP32 대용 아무 USB 장치 포함) 시 안정성	리셋 없음, get_throttled 0x0		

6. 블록 C (담당 3): UWB DWM1001-DEV 2노드 거리 측정

6.1 개념 정리: 어느 노드가 차량에, 어느 노드가 보호자에게?

DWM1001-DEV는 Qorvo(구 Decawave)의 **PANS 펌웨어**가 기본 탑재되어 있고, 노드를 **Anchor**(고정 기준점) 또는 **Tag**(움직이는 대상)로 설정하면 둘 사이 TWR(Two-Way Ranging) 거리를 잰다. 중요한 점은 **거리값이 출력되는 곳은 Tag 쪽 시리얼 셸**이라는 것이다. 따라서:

- **차량측 노드(Pi에 USB로 연결) = TAG**: 셸에서 거리값을 읽어 FSM 입력으로 쓴다.
- **보호자측 노드(보조배터리) = ANCHOR (initiator)**: 보호자가 들고 다니기만 하면 된다. 출력 읽을 필요 없음.
- 보고서/발표에서는 '차량측 노드 / 보호자측 노드'로 표기하면 Anchor·Tag 용어 혼선을 피할 수 있다. (심사 코멘트의 '보호자측 Tag도 독립 전원' 취지는 동일하게 충족.)
- Tag는 앵커가 1개뿐이어도 **그 앵커까지의 거리**를 출력한다(좌표 계산에는 3개 이상 필요하지만 우리는 거리만 쓴다). 첫 테스트에서 거리가 안 뜨면 6.2절 설정을 다시 확인.

6.2 설정 방법 (둘 중 하나, 시리얼 셸 권장)

방법 A: USB 시리얼 셸

1. 노드를 micro-USB로 노트북에 연결(J-Link CDC 포트가 잡힘). Tera Term/PuTTY로 **115200, 8N1** 접속.
2. **Enter를 두 번** 눌러 셸 모드 진입('dwm>' 프롬프트). `si` 로 시스템 정보 확인.
3. 보호자측 노드: `nmi` (anchor initiator 모드) → 재부팅됨.
4. 차량측 노드: `nmt` (tag 모드) → 재부팅됨.
5. 두 노드 모두 같은 네트워크 ID: `nis 0x1234` (임의 값, 둘이 같으면 됨). 필요 시 `aurs 1 1`로 태그 업데이트 주기를 100ms로 (기본값도 충분).
6. 차량측(Tag) 셸에서 `les` 또는 `lec`(CSV 형식) 입력 → 앵커 ID와 거리(m)가 주기적으로 출력되면 성공. 같은 명령을 다시 치면 출력 정지.

방법 B: Android 'DRTLS Manager' 앱 (BLE)

- Qorvo(Decawave) DRTLS Manager 앱으로 각 노드를 검색 → 네트워크 생성 → 노드별 Anchor/Tag 지정. 앵커 좌표는 (0,0,0)으로 두면 된다.
- 앱은 설정 확인용으로 편하지만, 최종적으로 Pi가 읽을 것은 어차피 **시리얼 셸의 lec 출력**이므로 방법 A도 익혀 둔다.

6.3 보호자측 노드 전원: 보조배터리 검증

- 보조배터리(No.35) → micro-USB 케이블 → DWM1001-DEV micro-USB. 보드 전원 LED 점등 확인.
- **문제 가능성**: DWM1001은 수십 mA만 먹어서 많은 보조배터리가 '기기 없음'으로 판단하고 **1~2분 뒤 자동 차단**된다. 반드시 **10분 이상 켜 두고 꺼지지 않는지** 확인.
- 꺼지면 대안: ① 보조배터리의 '저전류(소형기기) 모드' 버튼 확인 ② 다른 보조배터리로 교체 ③ USB 더미 부하(LED 조명 등)를 병렬로 물려 전류를 올림 ④ DWM1001-DEV의 배터리 커넥터에 3.7V LiPo 연결(리튬 취급 주의, 최후 수단).
- 실측 시 보호자측 노드는 사람이 **손에 들거나 가방 바깥**에 둔다. 몸이 안테나를 가리면 거리 오차가 커진다.

6.4 거리 테스트 절차 (5m·10s 이탈 판정 기준 검증)

1. 줄자로 0.5, 1, 2, 3, 5, 7 m 지점을 표시. 실내는 벽 반사(멀티패스)로 값이 튀므로 **긴 복도나 야외**에서 한다.
2. 차량측 노드는 노트북에 연결하고 책상 위 1m 높이에 고정, `lec` 출력 시작.
3. 보호자측 노드를 각 지점에서 20초 정지 → 출력된 거리의 평균·표준편차 기록.
4. 5m 지점에서 10초 이상 연속으로 5m 초과가 나오는지(보고서 기준: 5m 이상 10s 유지, 이동평균 7점) 확인. 판정오차(실거리 vs 측정)를 표로 남기면 심사 성능지표 '보호자 이탈 판정오차'에 바로 쓴다.
5. 시야 가림(사람 몸, 차량 문 닫힘) 조건에서도 한 번 재서 오차 증가량을 기록한다.

실거리(m)	평균 측정(m)	표준편차	오차(m)	비고(LOS/NLOS)
0.5				
1				
2				
3				
5				
7				

6.5 Pi 5 연결

```
ls /dev/ttyACM*     # DWM1001은 ttyACM 으로 잡힘 (mmWave와 번호가 섞이므로 udev 규칙으로 고정 권장)
# pyserial 115200 → CR 2회 전송(셸 진입) → 'lec' + CR 전송 → 'DIST,...' 행 파싱 → distance_m 로깅
```

참고 mmWave와 UWB가 모두 `/dev/ttyACM*`로 잡혀 부팅마다 번호가 바뀔 수 있다. 통합 단계에서 `udevadm info`로 시리얼 번호를 확인해 `/etc/udev/rules.d/`에 `/dev/uwb`, `/dev/mmwave_cli` 같은 고정 이름을 만든다.

6.6 블록 C 완료 조건 표

#	확인 항목	기대값	실측/결과	확인자
C1	두 노드 si 출력, 같은 네트워크 ID	모드 TAG / ANCHOR-INIT		
C2	차량측 lec 출력에 거리 표시	1Hz 이상 갱신		
C3	거리 오차표(6.4절)	LOS 오차 ±0.3m 내		
C4	보조배터리 10분 유지	꺼지지 않음		
C5	Pi에서 파이썬으로 거리 파싱	distance_m CSV 생성		

7. 4단계 (다같이): 통합 연결과 전원 투입 순서

7.1 모형차 내 물리 배치

부품	위치	고정 방법	이유
Pi 5 + ESP32/MOSFET 기판	트렁크 또는 뒷좌석 바닥 (아크릴 판 위)	스탠드오프 + 벨크로	케이블이 가장 짧게 모이는 곳, 발열 배출
Camera Module 3	룸미러 위치, 뒷좌석 향함	스탠드오프·벨크로, 흔들림 없이	프레임 차분은 흔들림에 민감
mmWave ODS	천장 중앙, 아래 향함	벨크로 + 케이블 그랜드로 선 고정	ODS는 오버헤드 장착 설계
PIR	천장 뒤쪽 또는 B필러, 뒷좌석 향함	벨크로, 렌즈 시야 마스킹	창밖 오검출 방지
SHT31	뒷좌석 헤드 높이, 팬 바람 직접 안 맞는 곳	벨크로	온도 예측 입력의 대표성
팬 ×2	창문 개구부 또는 천장 배기 위치	벨크로/케이블타이	환기 시연 가시성
부저	대시보드 하부	벨크로	소리가 밖으로 들리게
UWB 차량측 노드	대시보드 중앙, 안테나가 창 방향	벨크로	보호자 방향 LOS 확보
서보 (선택)	창문 스위치 옆 / 모형 창문 축	브래킷	플러스 알파
비상정지	모형차 외부, 손 닿기 쉬운 곳	판넬 홀 장착	심사위원 가시성
12V 입력·퓨즈·컨버터	트렁크, Pi 옆	케이블 그랜드로 인입선 고정	실차 시거잭에서 한 가닥으로 인입

7.2 연결 순서 (모두 전원 OFF 상태에서)

1. **GND 공통 확인**: 12V GND, 컨버터#1/#2 GND, ESP32 GND, MOSFET 소스, 서보 GND를 하나로 묶고 테스터기 연속성으로 확인.
2. **Pi USB 3개**: ESP32(micro-USB), UWB 차량측(micro-USB), mmWave(micro-USB). 전류가 큰 mmWave는 USB3(파란색) 포트에.
3. **카메라 CSI** 연결(5.2절).
4. **12V 입력선**을 메인 퓨즈에 연결(아직 어댑터는 꽂지 않음). 비상정지는 **눌러 둔 상태**(구동부 차단).
5. 컨버터#1 USB-A → USB-C 케이블 → Pi 5.

7.3 전원 투입 순서

1. 12V 어댑터(또는 시거잭) 투입 → 컨버터#1 5V 확인 → Pi 부팅(약 30초) → SSH 접속.
2. `ls /dev/ttyACM* /dev/ttyUSB*` → 장치 3개(ttyUSB0=ESP32, ttyACM×2=UWB, mmWave 2포트 → 실제 ttyACM 3개) 확인. `vcgencmd get_throttled` = 0x0.
3. ESP32 시리얼 열어 PIR/SHT31 JSON 수신 확인. UWB lec 거리 수신 확인. mmWave 프레임 수신 확인. 카메라 rpicam-still.
4. **비상정지 해제** → 구동 레일 12V 확인 → Pi에서 ESP32로 `FAN 1`, `BUZ 1` 명령 → 동작 확인 → OFF.
5. 종료: 구동부 OFF → `sudo shutdown -h now` → 녹색 LED 소등 후 12V 차단.

7.4 통합 체크리스트

#	항목	명령/방법	기대값	우선순위	결과
I1	전원 무결성	테스터기: 12V 입력, 5V ×2, 3.3V(ESP32)	12.0 / 5.0 / 3.3 ±5%	필수	
I2	Pi USB 장치 전부 인식	ls /dev/ttyACM* /dev/ttyUSB*	ttyUSB0 + ttyACM0~2	필수	
I3	USB 전류 여유	vcgencmd get_throttled	0x0	필수	

#	항목	명령/방법	기대값	우선순위	결과
I4	카메라	rpicam-still	뒷좌석 사진	필수	
I5	PIR/SHT31 JSON	cat /dev/ttyUSB0 (115200)	1초 주기 JSON	필수	
I6	UWB 거리	lec 파싱 스크립트	distance_m 갱신	필수	
I7	mmWave 프레임	파서 스크립트	프레임/초 > 5	필수	
I8	팬/부저 원격 구동	echo 'FAN 1' > /dev/ttyUSB0	동작	필수	
I9	비상정지	구동 중 누름	팬·부저 즉시 정지, Pi 유지	필수	
I10	서보	SRV1 명령	회전	선택	
I11	USB 탈착 시 감지	mmWave USB 뽑기	Pi가 장치 소실 인지	권장(FAIL_SAFE)	
I12	30분 연속 구동 안정성	전체 ON 상태 방치	리셋/과열 없음 (온도 < 70°C)	필수	

7.5 필수 시연 시나리오 하드웨어 리허설 (소프트웨어 FSM 전 단계)

심사 피드백의 필수 시연 흐름을 **사람이 수동으로 각 입력을 만들면서** 신호가 Pi 로그까지 도달하는지 확인한다. 이것이 되면 소프트웨어는 이 로그를 상태 전이로 엮기만 하면 된다.

단계	사람이 하는 행동	하드웨어에서 확인할 것	온도 시연 결정 사항
① 보호자 이탈	보호자측 노드를 들고 5m 이상 걸어감	UWB distance_m이 5m 초과로 10초 유지	-
② 1차 잔류 확인	뒷좌석에서 손·인형 움직임	PIR pir=1, 카메라 프레임 차분 픽셀 증가	PIR·카메라 AND/OR는 소프트웨어에서 확정(피드백 항목)
③ 존재 확인	뒷좌석에 사람/움직이는 대상	mmWave 포인트 수 임계 초과	-
④ 위험 온도 예측	SHT31에 손난로/드라이기 또는 **설정 온도값 주입**	온도 상승 곡선 로그	시연장에선 실측 35°C 도달이 어려우므로 '좌석별 사전 설정 온도 3종'을 소프트웨어 입력으로 허용(심사 코멘트 OK)
⑤ 능동 대응	-	팬 ON → 부저 ON 순서, LED	서보 창문은 별도 '추가 기능' 시연

7.6 소비전력 측정표 (심사 질문 대응: '소모 전력을 뭐로 비교하나')

테스터기 10A DC 전류 모드를 **12V 입력 +선에 직렬**로 넣고 각 상태에서 30초 평균값을 읽는다. 전력(W) = 12V × 전류(A). USB 전력측정기가 있으면 장치별(mmWave·UWB·ESP32) 5V 전류도 따로 잰다.

상태(FSM 단계)	ON인 장치	12V 입력 전류(A)	전력(W)	비고
STANDBY	Pi + ESP32 + UWB 차량노드			카메라·mmWave 비활성
UWB_DEPARTURE_CHECK	+ UWB 랭잉 활성			
PIR_CAMERA_CHECK	+ 카메라			
MMWAVE_CHECK	+ mmWave			
THERMAL_PREDICTION	= 위와 동일			
ACTUATOR_CONTROL	+ 팬 2 + 부저			
전부 상시 ON (비교군)	모든 장치			Cascading 대비 절감률 계산용

부록 A. 핀맵·커넥터 총괄표

연결	From	To	케이블/커넥터	블록
Pi ↔ 카메라	Pi 5 CAM1 (22핀)	Camera Module 3 (15핀)	22↔15핀 FPC 케이블 (추가 구매)	B
Pi ↔ mmWave	Pi USB3 포트	IWR6843ISK-ODS micro-USB	micro-USB 데이터 케이블	B
Pi ↔ UWB 차량측	Pi USB2 포트	DWM1001-DEV micro-USB	micro-USB 데이터 케이블	C
Pi ↔ ESP32 #1	Pi USB2 포트	ESP32 DevKitC micro-USB	micro-USB 데이터 케이블	A/통합
Pi 전원	컨버터#1 USB-A (5V)	Pi 5 USB-C	USB-A→USB-C 케이블 (또는 27W 어댑터, 벤치)	A
ESP32 ↔ PIR	3V3 / GND / GPIO27	VCC / GND / OUT	20AWG 실리콘 와이어 + JST	A
ESP32 ↔ SHT31	3V3 / GND / GPIO21 / GPIO22	VCC / GND / SDA / SCL	4선 + JST	A
ESP32 → Q1(팬)	GPIO25 → 220Ω → 게이트	IRLZ44N G / D→팬 / S→GND	만능기판	A
ESP32 → Q2(부저)	GPIO26 → 220Ω → 게이트	IRLZ44N G / D→부저 / S→GND	만능기판	A
ESP32 → 서보1/2	GPIO18 / GPIO19	서보 신호(주황)	듀폰	A
서보 전원	컨버터#2 5V / GND	서보 빨강 / 갈색	JST 분기	A
ESP32 ← 비상정지(선택)	GPIO32(풀업) / GND	NO 접점 두 단자	2선	A
12V 입력	시거잭 or 12V 어댑터	퓨즈홀더 10A	20AWG 빨강/검정	A
12V 구동 레일	퓨즈 10A → 비상정지 NC → 퓨즈 5A	팬+, 부저+, 컨버터#2 IN	20AWG	A
UWB 보호자측 전원	보조배터리 USB	DWM1001-DEV micro-USB	micro-USB	C

부록 B. 부품 인벤토리 체크표 (엑셀 36항목)

No	품목	수량	블록	확인	메모(누락/파손)
1	라즈베리파이5 16GB	1	B		
2	Pi Camera Module 3	1	B		
3	IWR6843ISK-ODS mmWave	1	B		
4	DWM1001-DEV (UWB)	2	C		
5	실습용 납땜세트 D	1	A		
6	100×100 양면 만능기판	4	A		
7	돋보기/LED 집게 스탠드	1	A		
8	수축튜브 280세트	1	A		
9	절연 작업 매트	2	A		
10	DS3218 Pro 서보	2	A		
11	SHT31 온습도 (SEN0331)	1	A		
12	SparkFun PIR EKMC4607112K	1	A		
13	ESP32 DevKitC WROOM-32D	2	A		
14	12V 능동 부저 FQ-015	2	A		

No	품목	수량	블록	확인	메모(누락/파손)
15	1N5819 쇼트키	5	A		
16	디지털 테스터기 DM-1010	1	A(공용)		
17	LED 120mm 쿨링팬	1	A		
18	DC팬 108F 12V	1	A		
19	비상정지 CRE-30M1R	1	A		
20	차량용 시거잭 충전기	1	A		
21	Pi 27W USB-C 어댑터	1	B		
22	IP68 케이블 그랜드 PG7	9	통합		
23	탈부착 벨크로(듀얼락)	3	통합		
24	20AWG 3색 실리콘 와이어	1	A		
25	벨크로 케이블타이	3	통합		
26	JST 커넥터 키트	1	A		
27	M3×30 스탠드오프 키트	1	B/통합		
28	microSD 128GB + 리더기	1	B		
29	카퓨즈 10종 세트	1	A		
30	자동차 퓨즈홀더	5	A		
31	IRLZ44N MOSFET	2	A		
32	아크릴 순간접착제	1	통합		
33	막대저항 샘플키트	1	A		
34	12V→5V 10A 컨버터	2	A		
35	보조배터리 10,000mAh	1	C		
36	USB-A→micro 5핀 케이블	1	C		

부록 C. 문제 해결 FAQ

증상	가장 흔한 원인	조치
ESP32 업로드 실패 'Failed to connect'	자동 부팅모드 진입 실패, 충전전용 케이블	BOOT 버튼 누른 채 업로드 시작 / 데이터 케이블로 교체 / CP2102 드라이버
팬이 부팅 중 잠깐 돈다	게이트 풀다운 없음	게이트-소스 10kΩ 확인
MOSFET이 뜨겁다	게이트 전압 부족(3.3V 미도달) 또는 부하 과다	게이트 배선·GND 공통 확인, 부하 전류 측정
전원 넣자마자 퓨즈 단선	다이오드 역방향 또는 +/ 쇼트	1N5819 띠 방향(+12V 쪽) 확인, 저항 모드로 쇼트 점검
서보 동작 시 ESP32 리셋	서보를 보드 5V에서 급전	컨버터#2로 분리 급전, GND 공통
PIR이 항상 1 또는 항상 0	워밍업 미완, 출력 극성 반대, 시야에 창밖 움직임	30초 대기 / 코드 반전 / 렌즈 마스킹
SHT31 'not found'	SDA/SCL 바뀜, 주소 0x45	핀 교체, 스캐너 스케치로 주소 확인
카메라 목록에 없음	케이블 방향/22핀 케이블 아님/DISP 포트	5.2절 순서대로 재확인
mmWave COM 포트 안 잡힘	충전전용 케이블, 드라이버	데이터 케이블, XDS110 드라이버(UniFlash 설치 시 포함)

증상	가장 흔한 원인	조치
mmWave Visualizer 'no data'	SOP가 플래싱 모드 그대로, 포트 바뀜	Functional 모드로 전환 후 USB 재연결, CLI/Data 포트 순서 확인
Pi에서 USB 장치가 사라졌다 나타남	USB 전류 제한(600mA)	usb_max_current_enable=1, 전원 허브
UWB lec에 거리 없음	네트워크 ID 불일치, 두 노드 모두 Tag/Anchor, 앵커 initiator 아님	nis 재설정, nmi/nmt 역할 재확인
UWB 거리 튐(±1m)	실내 멀티패스, 안테나 가림	복도/야외 재측정, 이동평균 7점 적용
보조배터리가 꺼짐	저전류 자동 차단	6.3절 대안
ttyACM 번호가 부팅마다 바뀜	USB 열거 순서	udev 규칙으로 고정 이름

부록 D. 참고 자료

- Raspberry Pi 5 문서(카메라 연결, config.txt, rpicam-apps): raspberrypi.com/documentation
- TI IWR6843ISK-ODS User's Guide / mmWave Industrial Toolbox (Resource Explorer) / mmWave Demo Visualizer / UniFlash: ti.com
- Qorvo DWM1001-DEV: 'DWM1001 Firmware API Guide'(셸 명령 les/lec/nmi/nmt/nis), 'DRTLS Manager' 앱: qorvo.com
- IRLZ44N 데이터시트(로직 레벨, Vgs(th) 1~2V), 1N5819(1A 쇼트키), DS3218(4.8~6.8V, 스톨 ≥2A), EKMC4607112K(Panasonic PaPIRs), SHT31(I2C 0x44/0x45)
- 팀 저장소 hardware/ 폴더: 기존 Pi 4B 프로토타입 로거(pi_hardware_logger.py)는 GPIO 직결 구조이므로 ESP32 시리얼 JSON 입력으로 수정 예정(소프트웨어 단계).

문서 끝. 각 블록 담당자는 완료 조건 표를 채워 팀 채팅에 공유하고, 통합 일정은 세 블록 모두 완료 조건을 만족한 뒤 잡는다.